

BENEVOL

Security Audit Report

Community Donation & Sharing Platform — Flask / SQLite Application

Scope	Full application code review: Flask routes, SQLite queries, Jinja2 templates, session/auth, file uploads, REST API
Method	Manual source review + targeted exploit verification (live payloads tested against a running instance)
Fixed during this audit	1 vulnerability (Stored XSS) — verified with real attack payloads
Open findings	16 issues: 5 High, 6 Medium, 5 Low

Prepared by: Claude (Anthropic) — acting as an AI coding assistant on this project
Codebase: E:\DAANLOOP_SOFTWARE_ENGINEERING

Executive Summary

This report documents a full manual security review of the BENEVOL codebase, covering authentication, database access, session handling, file uploads, the REST API, and every Jinja2 template that renders user-supplied content. Findings were verified against a running instance of the app wherever practical (e.g. actual XSS payloads were submitted and the rendered HTML inspected), rather than relying on static code reading alone.

One critical vulnerability — a Stored Cross-Site Scripting (XSS) flaw in the Wishes and Donation Campaign detail pages — was found to be actively exploitable and has already been **fixed and verified** as part of this engagement. Sixteen further issues remain open and are catalogued below, ordered by severity.



Already Fixed During This Audit

FIXED

F1. Stored XSS in Wish & Campaign descriptions

Where: templates/sprint1/view_wish.html and templates/sprint1/view_campaign.html.

Issue: The wish/campaign description was rendered with `{{ description|replace('\n', '<br>')|safe }}`. The `|safe` filter disabled Jinja2's automatic HTML escaping, so any HTML a user typed into a wish or campaign description — including `<script>` tags — was rendered and executed as live HTML/JavaScript in every visitor's browser, including administrators.

Impact: Any registered user could plant a script that runs in an admin's browser when they review the campaign/wish — enabling session-cookie theft and full account takeover of the admin account.

Fix applied: Added a dedicated `n2br` Jinja filter in `app.py` that HTML-escapes the text via `markupsafe.escape()` on a plain Python string first, then converts real newlines to `<br>` tags, then marks the result Markup-safe. This avoids a subtle Jinja pitfall where chaining the built-in `replace` filter after `|e` re-escapes the inserted `<br>` tags themselves.

Verified with live payloads against the running app:

- Input: `line one<script>alert(document.cookie)</script>\nline two`
- Rendered output: text is HTML-escaped (script does not execute), while the `<br>` line breaks still render correctly for legitimate content.
- Regression-tested: all real seeded wishes/campaigns, and the full page suite (home, browse, donate, wishes, dashboard, profile, messages) all return HTTP 200 with no behavior change.

High / Critical Severity — Open

These issues are either actively exploitable with no special access, or would cause total compromise (full admin takeover) under realistic deployment conditions. Recommended to fix before any public or investor-facing deployment.

HIGH

1. Admin credentials printed on the public login page

Where: `templates/sprint1/login.html`.

The login page displays `Demo: admin / admin123 · demo / demo123` to any visitor, logged in or not.

Impact: Anyone who opens the login page can log in as the admin account with zero effort. If deployed as-is, the platform's admin account is compromised from the moment it goes live.

Recommendation: Remove the hint entirely before any non-local deployment, or gate it behind a `DEBUG/development-only` flag so it never renders in production.

HIGH

2. Unauthenticated REST API leaks private phone numbers

Where: `sprint3/api.py` — `GET /api/listing/<id>` and `GET /api/listings`.

These endpoints return the entire listing row (via `dict(row).jsonify`) with no authentication check, including the owner's `contact_number`.

The web UI (`view_listing.html`) deliberately hides this same phone number behind a login wall ("Login to see the contact number"). The API completely bypasses that protection.

Verified: Called `GET /api/listing/1` from a logged-out test client and received the owner's real contact number in the JSON response.

Recommendation: Require authentication on these endpoints (or strip `contact_number` from the serialized response for anonymous/non-owner requests, mirroring the web page's own rule).

Verified: `GET /api/listing/1` (no session cookie) → 200 OK, response JSON contains `"contact_number": "9876543210"`.

HIGH

3. No CSRF protection on any state-changing form

Where: application-wide — login, register, delete listing/wish/campaign, send message, submit report, admin review actions, ratings, bookmarks.

No CSRF token library (e.g. Flask-WTF's `CSRFProtect`) is installed, and no form includes a hidden anti-CSRF token.

Impact: A malicious external web page can silently submit a hidden form to any of these endpoints using a logged-in victim's browser session — e.g. deleting their listing, sending spam messages on their behalf, or (if an admin visits the malicious page) approving/removing arbitrary reported content.

Recommendation: Add Flask-WTF's `CSRFProtect(app)` and a `{{ csrf_token() }}` hidden field to every POST form (roughly 15 templates).

HIGH

4. Debug mode enabled in the app's run configuration

Where: `app.py` — `app.run(debug=True, ...)`.

If this configuration is ever used for a live/public deployment, any unhandled exception exposes a full stack trace and Werkzeug's interactive in-browser debugger console to visitors.

Impact: The debugger console allows arbitrary Python code execution on the server if reached (it is PIN-protected by default, but the PIN can sometimes be derived from predictable machine identifiers). Stack traces alone also leak file paths, source code, and configuration values.

Recommendation: Set `debug=False` for any deployment outside local development, and drive it from an environment variable rather than a hardcoded literal.

HIGH

5. Hardcoded fallback SECRET_KEY

Where: `config.py` — `SECRET_KEY = os.environ.get('SECRET_KEY', 'daanloop-secret-key-change-in-production')`.

If the app is deployed without explicitly setting the `SECRET_KEY` environment variable, it silently falls back to this hardcoded, publicly-visible string.

Impact: Flask signs session cookies with this key. Anyone who knows the fallback value (visible in this source code) can forge a session cookie asserting `role=admin` for any user id, bypassing the login system entirely.

Recommendation: Fail startup loudly if `SECRET_KEY` is not set in production, rather than silently falling back to a known value.

Medium Severity — Open

MEDIUM

6. No brute-force protection on login

Unlimited password attempts are allowed on `/login` — no lockout, delay, or CAPTCHA after repeated failures.

Recommendation: Add rate limiting (e.g. Flask-Limiter) keyed by username/IP, with exponential backoff or temporary lockout after ~5 failed attempts.

MEDIUM

7. No rate limiting on registration or content creation

Registration, listing/wish/campaign creation, and messaging have no throttling.

Impact: A script could mass-create fake accounts or flood the platform with spam listings, wishes, campaigns, or messages.

Recommendation: Apply per-IP/per-account rate limits to these endpoints.

MEDIUM

8. Session cookie missing Secure and SameSite flags

`SESSION_COOKIE_SECURE` and `SESSION_COOKIE_SAMESITE` are left at Flask's insecure defaults (not enforced, not set).

Impact: The session cookie can be transmitted over plain HTTP (susceptible to network interception) and sent along with cross-site requests, compounding the CSRF risk in finding #3.

Recommendation: Set `SESSION_COOKIE_SECURE=True` (once served over HTTPS) and `SESSION_COOKIE_SAMESITE='Lax'` in `config.py`.

MEDIUM

9. Admin account is publicly identifiable

Where: `GET /user/<id>` (public profile) requires no login and displays a red "ADMIN" role badge.

Impact: An attacker can browse sequential user IDs to find exactly which account is the administrator, then focus password-guessing or phishing effort on that one account.

Recommendation: Do not reveal the `role` field on public-facing profile pages.

MEDIUM

10. State-changing action exposed via GET request

Where: `GET /messages/<id>/read` marks a message as read as a side effect of a simple page view.

Impact: Because it's a GET (not POST) endpoint, it can be triggered invisibly — e.g. by an `` tag on another site, link-preview bots, or email/antivirus link-scanners — without any user interaction or confirmation.

Recommendation: Convert to a POST-only action, or explicitly accept this as a low-impact read-state toggle if changing it isn't practical right now.

MEDIUM

11. JSON API bypasses upload validation

Where: `sprint3/api.py` — `POST /api/create`.

The web form validates uploaded images through `save_upload()/allowed_file()`. The JSON API instead accepts an arbitrary `image_path` string directly from the request body with no validation against the same rules.

Recommendation: Route API-supplied image references through the same validation used by the web form, or reject arbitrary path values outright.

Low Severity / Hardening

LOW

12. Weak password policy

Only a 6-character minimum length is enforced at registration (`sprint1/auth.py`); no complexity requirement (mixed case, digits, symbols).

Recommendation: Enforce a stronger minimum (e.g. 8+ characters) and consider checking against common-password lists.

LOW

13. No email verification on signup

Any email address can be used to register without proving ownership of it.

Recommendation: Send a verification link before activating new accounts, particularly once email is used for password resets or donation receipts.

LOW

14. No security response headers configured

No `X-Frame-Options`, `Content-Security-Policy`, or `X-Content-Type-Options` headers are set anywhere in the app.

Impact: No defense-in-depth against clickjacking (embedding the site in a hidden `iframe`) or MIME-type-sniffing attacks, on top of whatever the browser does by default.

Recommendation: Add `Flask-Talisman` or manually set these headers in an `after_request` hook.

LOW

15. File uploads validated by extension only

`utils/helpers.py` — `allowed_file()` checks only the file extension, not the actual file content/magic bytes.

Practical risk is low here since Flask's static file server never executes uploaded files, but a renamed non-image file would currently pass the check.

Recommendation: Optionally verify actual image content (e.g. with Pillow's `Image.open().verify()`) before saving.

LOW

16. Repository/deployment hygiene (not yet applicable)

The project is not yet under version control. When a Git repository is initialized, ensure `daanloop.db` (contains real password hashes and user data) and any future `.env` file are added to `.gitignore` before the first commit, so they are never pushed to a public or shared remote.

What Is Already Solid

These areas were specifically checked and found to already follow good security practice — no changes recommended:

- **No SQL injection anywhere.** Every database query across every model file uses parameterized ? placeholders for user-supplied values. The handful of f-strings found only assemble WHERE-clause fragments from a fixed, hardcoded set of column names — never raw user input — including in the newer polymorphic reports/messages queries.
- **Passwords are hashed correctly** using Werkzeug's salted `generate_password_hash`.
- **Ownership checks are consistent** — every delete/edit route checks `owner_id == session['user_id']` or `role == 'admin'` before acting.
- **All `/admin/*` routes** are correctly protected with the `@admin_required` decorator; no missing coverage was found.
- **Mass-assignment is prevented** on profile/listing updates — both `update_user()` and `update_listing()` restrict writes to an explicit column whitelist, and callers pass named arguments rather than raw request dictionaries.
- **No directory listing** is exposed on the uploads folder (verified: `GET /static/uploads/` returns 404, not a file index).
- **No use of `eval`, `exec`, `pickle`, `subprocess`, or `render_template_string`** anywhere in application code (confirmed by a full-repository search) — no remote code execution or template-injection surface.

Suggested Priority Order

Given this application is intended for demonstration to judges/investors and eventual real users, the recommended order of remediation is:

- 1 Remove the on-page admin/demo credentials hint (#1) and fix the API contact-number leak (#2) — both are trivial to fix and are the most damaging if discovered during a live demo.
- 2 Set `debug=False` and enforce a real `SECRET_KEY` (#4, #5) before any deployment outside your own machine.
- 3 Add CSRF protection (#3) — broader change (touches ~15 templates) but essential before public launch.
- 4 Add login rate-limiting and cookie hardening (#6, #8).
- 5 Address the remaining medium/low items opportunistically as the app matures.

This report reflects a manual code review at a point in time and is not a substitute for a professional penetration test or a formal security audit prior to handling real payment data.